CP2406

Explaining the Galaxy Simulation Project to C++ Beginners

A plain-English teaching guide for introducing the project before showing the code

Teaching approach: Begin with a simple model of stars moving under gravity. Then show the program flow and introduce each C++ component as a worker with one responsibility. Students do not need to derive the Barnes-Hut mathematics.

1. Start with the Story

Imagine trying to simulate a galaxy containing many moving bodies. Every body attracts other bodies, so the program must repeatedly calculate forces, update movement and draw the new positions. Checking every possible pair becomes expensive as the number of bodies grows.

The supplied simulation already uses a Barnes-Hut tree to organise bodies in space and approximate gravitational interactions. The student task is to improve the surrounding C++ software: configuration, safe collision merging, experiment reporting, testing and program organisation.

Important scope: This is an educational software model. Students are assessed on C++ design and testing, not astrophysics expertise.

2. Show the Complete Data Flow

```
Command-Line Options
        |
        v
Validated Simulation Configuration
        |
        v
Create the Bodies
        |
        v
Build the Barnes-Hut Octree
        |
```

```
        v
Approximate Gravitational Forces
        |
        v
Sample Possible Collision Pairs
        |
        v
Merge Bodies When Required
        |
        v
Update Velocity and Position
        |
        v
Render Frames and Calculate Metrics
        |
        v
Write Experiment CSV and Video
```

Show this pipeline before discussing source code. It gives beginners a map of where settings enter, how the simulation advances, and what evidence is produced.

3. Explain the Main Components as Team Members

SimulationConfig

Simple description: Stores and checks the experiment settings.

- Holds body count, system size, number of steps, time step, collision threshold, sample count, seed and output filename.
- Reads command-line options.
- Rejects invalid values with clear messages.

Types

Simple description: Defines the project's basic data records.

- vec3 stores x, y and z values.
- body stores position, velocity, acceleration and mass.
- Operator functions allow readable vector addition, subtraction, multiplication and division.

Octant

Simple description: Represents one cubic region of space.

- Stores a centre and length.
- Checks whether a position lies inside the region.
- Creates the eight smaller child regions.

Bhtree

Simple description: Organises bodies and approximates gravity.

- Stores a Barnes-Hut tree node.
- Places bodies into child octants.
- Stores combined mass and centre-of-mass information.
- Uses the tree to approximate gravitational interactions.

Collisions

Simple description: Handles safe body merging.

- Rejects negative thresholds, self-pairs and retired bodies.
- Checks the distance between two bodies.
- Conserves total mass and calculates mass-weighted position and velocity.
- Runs a bounded random sampling pass and returns statistics.

Simulation

Simple description: Runs the experiment.

- Initialises bodies from the configuration and seed.
- Builds the tree and calculates acceleration for active bodies.
- Runs collision sampling.
- Updates velocity and position, and calculates summary metrics.

Renderer

Simple description: Turns active body positions into image frames.

- Projects positions onto a two-dimensional image.
- Skips retired bodies.
- Writes PPM snapshot files.

CsvReporter

Simple description: Writes one experiment row.

- Records configuration, seed, collision counts, active bodies, total mass, barycentre and elapsed time.

main.cpp

Simple description: Coordinates the program.

- Parses configuration, clears old frames, runs the simulation, writes the CSV and requests video creation.
- Handles top-level errors and progress messages.

4. Introduce C++ Concepts Through Familiar Comparisons

Struct = a labelled record

```
struct body {
    vec3 position;
    vec3 velocity;
    vec3 accel;
    double mass;
};
```

A body is a record containing all information needed to represent one simulated object.

Vector = a collection of bodies

```
std::vector<body> bodies;
```

The vector stores all bodies and can be traversed with a loop.

Pointer = an address of an existing body or tree node

```
body* a
a->position.x is the same as (*a).position.x
```

The arrow operator accesses a member through a pointer. The tree also uses pointers to its eight possible child nodes.

Reference = work with the original object

```
body& a
```

A reference allows a collision function to update the original body rather than a copy.

Random seed = repeatable random choices

```
std::mt19937 generator(seed);
```

Using the same starting configuration and explicit seed supports repeatable collision-pair selections, which is useful for controlled experiments.

5. Explain the Octree as Dividing a Box

Imagine one large cube containing the galaxy. Divide it into eight smaller cubes: four above and four below. If a smaller region still contains multiple bodies, it can be divided again.

```
                LARGE CUBE
              /       |       \
       eight smaller spatial regions
       each region may divide again
```

The Octant class describes one cube. The Bhtree class connects these cubes into a tree. This gives the simulation a spatial organisation instead of treating the bodies as one unstructured list.

> Beginner message: The tree is a filing system for space. Nearby bodies are stored in detailed regions, while a sufficiently distant group can be treated using its combined mass and centre.

6. Explain One Simulation Step

1. Create a root region that contains the active bodies.
2. Insert each active body into the Barnes-Hut tree.
3. Reset each active body's acceleration.
4. Use the tree to calculate gravitational acceleration.
5. Sample a fixed number of possible collision pairs.
6. Merge eligible bodies safely.
7. Update velocity using acceleration.
8. Update position using velocity.
9. Render the new positions and continue to the next step.

7. Explain Collision Merging with Two Shopping Trolleys

Use a simple analogy: two loaded trolleys meet and become one. The combined trolley must contain the total load, and its resulting position and movement must account for how heavy each original trolley was.

```
total mass = mass A + mass B

merged position =
(position A x mass A + position B x mass B) / total mass

merged velocity =
```

```
(velocity A x mass A + velocity B x mass B) / total mass
```

The surviving body receives the total mass and mass-weighted position and velocity. The second body is retired by setting its mass to zero so later calculations ignore it.

Essential safety checks: do not merge a body with itself; ignore a retired body; reject a negative threshold; do not merge bodies farther apart than the threshold.

8. Explain Bounded Collision Sampling

The program does not need to check every possible pair. It samples a configured number of candidate pairs. The loop must be limited by attempts, not by successful merges.

```
Repeat exactly N attempts:
    choose two indices
    if they are the same, skip
    if either body is retired, skip
    otherwise count a valid pair
    merge if the distance rule is satisfied
```

If a loop waited for N successful collisions, it might never finish when collisions are rare or disabled. A fixed attempt count guarantees termination.

9. Separate the Supplied Physics from the Student's Main Work

```
Keep working
Barnes-Hut force approximation -> tree interaction -> motion update

Main assessment focus
configuration -> validation -> collision merge -> bounded sampling -> reporting ->
experiments -> testing and refactoring
```

This distinction helps beginners focus. They do not need to derive the gravitational model or redesign the Barnes-Hut approximation. They need to understand how to organise, validate, test and report a C++ simulation.

10. A Short Classroom Explanation

“This project simulates many bodies moving under gravity. It uses an octree to organise space so distant groups can be approximated efficiently. At each step, it calculates forces, samples possible collisions, safely merges eligible bodies, updates movement, draws a frame and records experiment results. The C++ learning is in structs, vectors, pointers, references, classes, random-number generators, validation, file output, testing and separating responsibilities.”

11. Recommended Teaching Sequence

10. Run the unchanged project and show one frame, the video and the CSV row.
11. Draw the complete data-flow diagram.
12. Explain vec3 and body as simple records.
13. Demonstrate one two-body merge on paper.
14. Show why the same body cannot be selected twice.
15. Explain attempts versus successful collisions.
16. Introduce the large cube and its eight child octants.
17. Connect each idea to SimulationConfig, Collisions, Simulation, Renderer and CsvReporter.
18. Test two manually created bodies before running the full galaxy.
19. Run a tiny fixed-seed collision sample before a full experiment.

12. What Beginners Do Not Need to Understand Immediately

- A derivation of the Barnes-Hut force approximation.
- Scientifically accurate galaxy modelling.
- Every gravitational constant and unit conversion.
- Advanced memory-management alternatives.
- Video encoding details inside FFmpeg.
- Performance optimisation before the basic tests pass.

13. Final Teaching Advice

Start with bodies, movement and one safe merge. Then build outward to collision sampling, repeated simulation steps, the octree and experiment reporting. Do not begin by reading Bhtree.cpp line by line. Once students understand the data journey and the reason for each component, pointers, loops and function calls have a clear purpose.

Best first test: Create two bodies with simple masses, positions and velocities. Calculate the expected total mass, centre-of-mass position and merged velocity by hand, then compare the

function result. Next test self-pair, retired body, outside-threshold and exact-threshold cases.